

Custom HashMap with String Keys

Hash Tables, Custom Hash Function, Separate Chaining, Rehashing · Mentor-Led Lab

Developed by TalenciaGlobal

Difficulty ★★★ Intermediate

Problem

Design a MyHashMap that maps string keys to integer values, implementing the hash table yourself - no built-in dict, HashMap or unordered_map. Collisions are resolved by separate chaining: each bucket holds a linked list of entries that hash to it.

Support put(key, value) (insert or update), get(key) (the value or -1), and remove(key). Keep the load factor at or below 0.75 by rehashing - doubling the capacity and re-inserting every entry - whenever an insertion pushes it past the threshold.

Learning objectives: understand why hash tables beat direct-address tables, build and tune a string hash function, resolve collisions by chaining, and manage the load factor with automatic rehashing so operations stay average $O(1)$.

Operations & Rules

- PUT key value - insert the pair, or update the value if the key already exists.
- GET key - output the mapped value, or -1 if the key is absent.
- REMOVE key - delete the key and its value if present.
- Hash: a polynomial rolling hash, $\text{hash} = (\text{hash} * 31 + \text{ord}(\text{char})) \bmod \text{capacity}$, applied over the key's characters.
- Load factor: rehash to double the capacity whenever $\text{size} / \text{capacity}$ exceeds 0.75 after an insertion. The map starts at capacity 16.

Input / Output Format

Input: Each line is one operation: PUT key value, GET key, or REMOVE key (read until end of input).

Output: One line per GET (its result), then Size and Capacity of the final table.

Examples

Example 1

Input: put(apple,3); put(banana,5); get(apple); get(mango); put(apple,10); get(apple);
remove(banana); get(banana)

Output:

```
3
-1
10
-1
```

Explanation: apple maps to 3 then is updated to 10; mango is absent (-1); after banana is removed it too returns -1.

Example 2

Input: put(x,1); get(x); remove(x); get(x)

Output:

```
1
-1
```

Explanation: x reads back as 1, then -1 once removed.

Constraints

- $1 \leq \text{key.length} \leq 20$; up to 10,000 operations.
- Implement your own hash table with chaining - no built-in map types.
- Average $O(1)$ per operation; rehash keeps the load factor ≤ 0.75 .